PHYS30762 Object Oriented Programming in C++

Assignment 5 Report

Kuba Kane
11019699

Department of Physics and Astronomy
The University of Manchester

April 2026

Contents

1 Program summary **1**
1.1 Electron class 1
1.2 Muon class 1
1.3 Tau class 2

2 References **3**

Note to markers: For version history please see the README file for different branches on the Github repository for assignment 5, as this contains planning and content for this report.

1. Program summary

This program is based around a `Lepton` class, which is an abstract base class from which `Electron`, `Muon`, `Tau`, and `Neutrino` are derived classes. The `Lepton` class contains a unique pointer to a `FourMomentum` object, whose class has energy, x momentum, y momentum, z momentum, and rest mass, as data members (all measured in MeV). The `Lepton` class also contains a rest mass and relative charge, as well as a boolean expression which if true represents an antilepton by flipping the charge data member. The `Lepton` class contains 2 virtual member functions which makes it abstract. The first is the `info()` function which prints out the aforementioned properties of the `Lepton` class, and `has_decayed()` which is true if the lepton has decayed.

The derived classes inheret all these properties, but also have their own data members and member functions. These additional properties are printed by overriding the `info()` function (using `base_info()` to print the properties from `Lepton`, and are also used to determine the outcome of `has_decayed()`. Figure X is a UML diagram which shows all of the properties of these leptons. All of the derived classes use `static constexpr` to store their mass and charge.

1.1. Electron class

This class introduces an interaction with a detector, primarily through calculating the shower depth in a tungsten (W) detector via the member function `shower_depth_in_tungsten()`. The equation for shower depth X used was

$$X = X_0 \frac{ln(E_0/E_c)}{ln2}, \tag{1}$$

where X_0 is the radiation length, E_c is the critical energy, and E_0 is the initial energy of the electron (from its `FourMomentum`). The first 2 are defined within the function, and are set values for tungsten. E_0 is accessed via the pointer to the `FourMomnetum` from the Lepton base class. `Electron` also has the function `shower_contained(const double detector_length_in)` which, given a detector length, determines if the shower is contained within the detector.

1.2. Muon class

The `Muon` class introduces `time_in_detector()` which can be specified in a `Muon` object which if it is greater than the constant `MUON_MEAN_LIFETIME` stored as a private data member, sets `has_decayed()` to be true. A `validate_time()` static function is used to check that the time entered by the user is not negative or greater than 1 second.

1.3. Tau class

The `Tau` class also uses `time_in_detector()` along with `TAU_MEAN_LIFETIME` to handle the `has_decayed()` function. Unlike the `Muon` classes, it takes into account the different decay modes of taus, given that 64% of the time its decay products are hadronic, 17.8% of the time they involve electrons, and 17.4% of the time they involve muons.

2. References

The following sources were used in the research for this assignment.

[1] https://www.geeksforgeeks.org/cpp/pure-virtual-functions-and-abstract-classes/

[2] https://en.cppreference.com/w/cpp/language/rule_of_three.html

[3] https://www.geeksforgeeks.org/cpp/understanding-constexper-specifier-in-cpp/

[4] https://www.geeksforgeeks.org/cpp/sqrt-sqrtl-sqrtf-cpp/

[5] https://www.geeksforgeeks.org/cpp/enum-classes-in-c-and-their-advantage-over-enum-datatype/

[6] https://www.geeksforgeeks.org/cpp/range-based-loop-c/

[7] https://en.wikipedia.org/wiki/Particle_decay

[8] https://physics.stackexchange.com/questions/282245/muon-decay-mode

[9] https://en.wikipedia.org/wiki/Muon

[10] https://en.wikipedia.org/wiki/Tau_(particle)

[11] https://tikz.net/sm_decay_matrix/

[12] https://www.geeksforgeeks.org/cpp/stduniform_real_-distribution-class-in-c-with-examples/

[13] https://www.w3schools.com/cpp/cpp_switch.asp

[14] https://www.geeksforgeeks.org/cpp/can-constructor-private-cpp/

[15] https://www.w3schools.com/cpp/cpp_constructors_overloading.asp

[16] https://www.geeksforgeeks.org/system-design/factory-method-pattern-c-design-patterns/

[17] https://pdg.lbl.gov/2020/AtomicNuclearProperties/HTML/tungsten_W.html

[18] https://en.wikipedia.org/wiki/Particle_shower

[19] https://indico.capa.unizar.es/event/23/sessions/48/attachments/287/393/COST-TS-Belgrade-Course5-Tutorial.pdf